

INSTRUÇÕES - LEIAM ANTES DE IR ÀS QUESTÕES!

- I. Leiam a prova com atenção e em silêncio. A avaliação tem 9 questões de valores diferentes.
- II. A prova é em dupla, **offline** e com consulta apenas ao material da dupla. Se caso for ao banheiro, deixe o celular na mesa do professor.
- III. Ao término da prova, **chame o professor para recolher seu arquivo zip com o conteúdo da prova, o arquivo deve ter o nome da dupla completo.**
- IV. A prova se inicia às 9h30m e terminará às 13h. Não serão aceitas entregas tardias. Tempo mínimo de permanência: 1h30m.
- V. Comentem todo o código explicando cada consulta. Código sem comentário explicativo será desconsiderado.
- VI. Dúvidas? Retorne ao primeiro item. A interpretação das questões faz parte da prova.

QUESTÕES

- 1) (0,5 ponto) Você está migrando o sistema de um e-commerce tradicional (SQL) para o MongoDB. No modelo relacional antigo, os dados do cliente ficavam em uma tabela clientes e seus endereços de entrega em uma tabela **enderecos**, exigindo *joins* constantes. Sabendo que o MongoDB possui um esquema flexível e suporta documentos embutidos (Embedded Documents), projete o novo formato. Escreva o comando **db.clientes.insertOne()** contendo a estrutura JSON ideal para armazenar o cliente e seus endereços em um único documento, otimizando a leitura sem a necessidade de junções externas Dados Relacionais (Referência):
 - Cliente: ID: 1, Nome: "Tech Solutions Corp", Setor: "B2B"
 - Endereço 1: Rua A, 100 - São Paulo
 - Endereço 2: Rua B, 200 - Rio de Janeiro
- 2) (0,5 pontos) Uma empresa de logística possui uma coleção remessas com milhares de pacotes em trânsito. Coleção Inicial (remessas):

```
[
  { "_id": 1, "destino": "SP", "peso_kg": 45, "status": "Pendente", "urgente": true },
  { "_id": 2, "destino": "RJ", "peso_kg": 60, "status": "Em Rota", "urgente": false },
  { "_id": 3, "destino": "SP", "peso_kg": 55, "status": "Pendente", "urgente": false }
]
```

O coordenador logístico precisa de um relatório específico. Escreva uma consulta utilizando o comando **.find()** e os operadores corretos para retornar os documentos em que o **peso_kg** seja maior ou igual a 50 E que satisfaçam pelo menos uma das seguintes condições: o **destino** é "SP" OU o status é "Pendente".

- 3) (0,5 ponto) Um sistema de carteiras digitais (FinTech) precisa realizar um reajuste e uma atualização em massa nos saldos de seus usuários ativos. Coleção Inicial (**contas**):

```
[
  { "_id": 101, "cliente": "João", "saldo": 1000, "status": "Ativa" },
  { "_id": 102, "cliente": "Maria", "saldo": 2500, "status": "Ativa" },
  { "_id": 103, "cliente": "Carlos", "saldo": 500, "status": "Inativa" }
]
```

Escreva o comando **db.contas.updateMany()** que localize apenas as contas com **status** igual a "Ativa" e realize duas operações simultâneas no documento:

- Multiplique o valor do saldo por 1.05 (rendimento de 5%) utilizando o operador matemático adequado.
- Defina um novo campo chamado **ultima_atualizacao** com o valor numérico 2026.

- 4) (1,0 ponto) O banco de dados de uma rede social armazena postagens e as tags (palavras-chave) associadas a elas em formato de Array. Coleção Inicial (**postagens**):

```
[
  {
    "_id": 505,
    "autor": "dev_guru",
    "conteudo": "Dicas de MongoDB",
    "tags": ["nosql", "banco_de_dados", "antigo"]
  }
]
```

Escreva um único comando **db.postagens.updateOne()** para o documento de **_id: 505** que execute duas ações simultâneas em seus arrays:

- Remova a tag "antigo" utilizando o operador específico de exclusão em arrays.
- Adicione as novas tags "performance" e "json" de uma só vez, garantindo a inserção múltipla adequada.

- 5) (1,0 ponto) Um sistema de prontuários médicos (HealthTech) possui uma coleção pacientes com mais de 5 milhões de registros. O sistema frequentemente executa a seguinte consulta para relatórios demográficos: **db.pacientes.find({ cidade: "Curitiba" }).sort({ ano_nascimento: -1 })**

- Escreva o comando para criar um Índice Composto que otimize perfeitamente essa consulta, considerando as etapas de filtragem e ordenação.
- Explique teoricamente qual comando você utilizaria para verificar se o MongoDB realmente adotou o seu índice na execução da consulta e qual métrica indicaria o sucesso da operação (redução de varredura completa).

- 6) (1,0 pontos) Você atua na área de análise de dados de um supermercado. A coleção vendas registra milhares de transações diárias. Coleção Inicial (**vendas**):

```
[
  { "_id": 1, "setor": "Hortifruti", "valor_total": 150, "ano": 2026 },
  { "_id": 2, "setor": "Açougue", "valor_total": 300, "ano": 2026 },
  { "_id": 3, "setor": "Hortifruti", "valor_total": 80, "ano": 2025 }
]
```

Monte uma pipeline de agregação composta por três estágios sequenciais:

- Filtre apenas as vendas referentes ao **ano** 2026.
- Agrupe os documentos pelo campo **setor**, calculando a soma do **valor_total** em uma nova variável chamada **faturamento**.
- Ordene o resultado final pelo **faturamento** de forma decrescente.

- 7) (1,0 pontos) Uma plataforma de streaming gerencia seu catálogo dividindo dados em duas coleções por questões de arquitetura: **filmes** e **avaliacoes**, sendo:

filmes

```
{ "_id": 10, "titulo": "O Retorno do NoSQL", "genero": "Sci-Fi" }
```

avaliacoes

```
{ "_id": 999, "filme_id": 10, "nota": 5, "comentario": "Excelente!" }
```

Utilizando o Aggregation Framework, escreva uma consulta a partir da coleção **filmes** que utilize o operador de chave estrangeira adequado para buscar e anexar as avaliações correspondentes de cada filme dentro de um novo campo chamado **feedbacks**.

- 8) (2,0 pontos) O setor de Recursos Humanos de uma corporação mantém uma coleção **funcionarios**, onde cada colaborador possui um array com as notas de suas avaliações de desempenho bimestrais.

```
[
  { "_id": 1, "nome": "Alice", "notas_bimestrais": [8, 9, 10, 7] },
  { "_id": 2, "nome": "Bob", "notas_bimestrais": [5, 6, 6, 7] }
]
```

Crie uma Pipeline avançada (**db.funcionarios.aggregate()**) que:

- Desestruture o array **notas_bimestrais**, criando um documento para cada nota.
- Agrupe os dados novamente pelo **_id** e nome do funcionário, calculando a média das notas utilizando o operador apropriado em um campo chamado **media_anual**.
- Adicione um campo projetado (novo campo) chamado **aprovado_para_bonus** contendo um valor booleano (**true** ou **false**). Utilize um Operador Condicional estruturado que verifique se a **media_anual** é superior a 8 para conceder o bônus.

- 9) (2,5 pontos) O sistema de um aplicativo de delivery possui os dados separados em duas coleções: restaurantes e pedidos. A equipe de finanças precisa saber o faturamento total gerado por cada restaurante em 2026.

restaurantes

```
{ "_id": "rest_1", "nome": "Pizzaria Bella", "categoria": "Italiana" }  
{ "_id": "rest_2", "nome": "Sushi Express", "categoria": "Japonesa" }
```

pedidos

```
{ "_id": "ped_901", "restaurante_id": "rest_1", "valor": 80.50, "ano": 2026 }  
{ "_id": "ped_902", "restaurante_id": "rest_1", "valor": 120.00, "ano": 2026 }  
{ "_id": "ped_903", "restaurante_id": "rest_2", "valor": 65.00, "ano": 2026 }
```

Escreva uma pipeline de agregação iniciada na coleção **restaurantes** que:

- Realize o join com a coleção **pedidos**. Use as chaves corretas para o relacionamento.
- Agrupe os dados de forma a calcular o faturamento total por restaurante. O resultado deve exibir o nome do restaurante e a soma do campo valor dos pedidos.